

USpt: Updatable Signature with Public Tokens

Haotian Yin^a, Jie Zhang^{*a}, Wanxin Li^{*a}, Yuji Dong^b, Eng Gee Lim^a,
Dominik Wojtczak^c

^a*School of Advanced Technology, Xi'an Jiaotong-Liverpool
University, Suzhou, 215123, China*

^b*School of Internet of Things, Xi'an Jiaotong-Liverpool
University, Suzhou, 215400, China*

^c*Department of Computer Science, University of Liverpool, Liverpool, L69 3BX, England*

Abstract

The Updatable Signature (US) allows valid signatures to be updated by an update token without accessing the newly generated signing key. Cini et al. (PKC'21) formally defined this signature and gave several constructions. However, their security model requires the secrecy of the update token, which is not applicable in many common application scenarios where existing signatures have been distributed to many parties. In addition, one can use the same token to update both the signing key and signatures, and all signatures can be updated by a single token, whereas the adversarial signature generated by an adversary might also be updated.

This work explores the (im)possibility of constructing an Updatable Signature with public tokens (USpt). Specifically, we first define the updatable signature with public tokens and present its security model. Then, from considering existing US schemes, we found that a secure USpt must properly handle a transform function from signature-update token to key-update token. We further formally proved the impossibility of constructing a secure USpt if (1) there is no transform function between key pairs and signatures, or (2) the signature-update token can be derived from the public keys of adjacent epochs. Finally, we present a concrete USpt scheme based on the BLS signature.

Keywords: Updatable signature, Key-homomorphic signature, BLS signature

1. Introduction

A digital signature scheme is a cryptographic scheme that ensures information integrity, authentication, and non-repudiation. It allows the data sender to “sign” the information, and the receiver can verify the signature to confirm that the information is indeed from the claimed sender and has not been tampered with during transmission. In other words, a digital signature scheme can be designed to provide signature service through unforgeability under the circumstance that the adversary can acquire a list of message-signature pairs and is trying to forge a signature for some new message. Digital signature is an indispensable security tool in modern digital communications and file exchange and is widely used in software distribution, email, online transactions, and other scenarios that require high-security verification. A signature scheme’s security goals and threats may vary in different situations/real-world environments. They are formally modelled/defined in security models (or games) between a challenger and an adversary, where the adversary’s abilities are modelled as queries/access to oracles provided by the challenger, who can reveal some (possibly secret) information about the scheme.

Post-compromise security is a relatively new security goal for digital signatures. This security property was formally defined by Cohn-Gordon, Cremers, and Garratt [1], who considered the security of authenticated key exchange (AKE) protocols [2, 3] after a party was fully compromised under some reasonable assumptions. Informally, in their adversary model with post-compromise security through the state, there must be at least one uncompromised session before the test session (the session key of which is picked as a challenge). In other words, post-compromise security considers the security after full compromise but adds some restrictions to the adversary afterwards. A similar security goal is forward security, which requires that subsequent compromises will not affect the security of any previous sessions. A common feature of forward and post-compromise security is that they consider the security of cryptographic schemes in terms of time sequence.

Updatable signature (US) is defined as capturing post-compromise security with respect to signature schemes. Cini et al. [4] present the definition of US with secret tokens and its security model inspired by updatable encryption. Their model divides the operations of signers and verifiers into epochs, and in each epoch, different private keys are used for signing and public keys for verification. Suppose an adversary obtained the signing key sk_e of some

signer in epoch e , and then the adversary lost access to the signer’s local state (this attack can be considered as a transient corruption [5] without controlling the signer); the signer is somewhat aware of this leakage and wants to update its local signing key to sk_{e+1} , which also advances the epoch to $e + 1$. To keep the validity of previous signatures in epoch e , say a signature σ_e without loss of generality, the signer also generates a token Δ_{e+1} and *secretly* transfer it to the verifier who wants to update σ_e to a current-epoch-valid signature, σ_{e+1} . Interestingly, different from traditional signature schemes, this US definition introduces a trusted third-party *agent*, who is the receiver of the update token and responsible for updating the signatures instead of the verifiers.

1.1. Motivation

The US in [4] relies on a trusted third-party *agent*, which limits its application scenarios and might conflict with some system requirements, e.g., a trustless blockchain application. However, since the update token is secret, trivially removing the agent is also impractical, which will then need to securely transmit the update token to each signature holder/verifier and assume they are trusted. In addition, the update token can also be used to update the signing key, which means that if it is leaked, then the adversary can use it to update the signing key of the last epoch and continue forging valid signatures. Therefore, we seek to make the token *public* and avoid using a trusted third-party in US schemes to overcome the above limitations.

1.2. Related and Previous Work

The framework of Cini et al. [4] is well established for the US with *secret* tokens. They build their first construction based on Key-Homomorphic signature schemes [6], two features of which are the *homomorphism* of signing keys and the *perfect adaptability* of signatures (formally defined in 4). They also designed US from BLS signature [7] equipping a property called message independence (MI): the update algorithm does not need to take the message as an input. This feature simplifies the update procedure since the verifier does not need to retrieve the original message. The security model rules out trivial attacks by a *leakage profile* [8], compromised by key-update inferences, token inferences, and signature-token inferences. Those records are used to log the adversary’s compromising queries, therefore providing evidence to identify the challengable signatures and corresponding epochs.

Zhou and Liu [9] improved the US schemes by weakening the function of update tokens, which is similar to our goals, but their application scenario is the same as that of Cini et al. [4]. Specifically, their solution relies on a secure channel established by an AKE protocol [10] between the signer and a trusted agent (considered as a server in their work), while the secure channel and extra protocols are major issues we want to avoid and also, the deployment of an AKE protocol renders the final scheme interactive. Furthermore, their update process also utilised indistinguishability obfuscation [11, 12], introducing additional computational overhead that is considerable relative to a signature scheme.

Bellare and Miner [13] devised the first forward-secure signature scheme. This scheme updates the signing key without modifying the verification key. An updated key-signed signature can be verified by the verification key with the corresponding epoch “time-stamp”. However, this kind of signature scheme does not care about the signature update; the signer must sign the message again even though this message was signed in the previous epoch. This work might not be technically relevant to the US cause they have different goals, but the forward security is quite close to the post-compromise security. In their forward-secure signature scheme, there is no trusted third party to play the role of agent while updating the signature (the signer and the verifier follow the implicit epoch proceeding mechanism by the order of signatures). Our idea of the US with public tokens is aligned with this spirit.

1.3. Updatable Signature with Public Tokens

We informally introduce our definition of the US with public tokens (USpt) here. Recall the original demand for key updating: 1) replacing leaked keys; 2) meantime, existing signatures can be updated into valid new signatures using signature update tokens released by the signer so as to avoid resigning the messages using the updated signing key. The first requirement comes from the need for security, and the second requirement comes from the need for usability. Suppose there is some adversary $\mathcal{A}$ who has obtained a signing key sk_e , $\mathcal{A}$ is able to generate *valid* signatures, say σ'_e , in epoch e , but not the signature σ'_{e+1} valid in epoch $e + 1$ since the key is replaced from requirement 1). Because our US scheme publishes update tokens publicly, $\mathcal{A}$ can always use this token to update its forged signature σ'_e to a *valid* signature σ'_{e+1} satisfying the requirement 2). Note that here, we refer *valid* to the signatures that can be verified by the signer’s public key of the current

epoch. If we exclude this kind of attack in our security model, i.e., challenging a signature from updating a forged but expired signature, this definition is too weak for the practical scenarios: the adversary $\mathcal{A}$ is not even allowed to challenge the message chosen by itself.

In this case, we must prevent a public token from being exploited by adversaries. Our solution is under the assumption of a *signed-message database*: the signer maintains a database to record the messages (or hash of them) it signed for future updating the signatures that itself generated. Therefore, the signer is able to *distinguish* the signatures generated by itself from those forged by other parties even if its signing key is leaked, and the published tokens are only for signatures generated by itself. This assumption is not too strong for the application of US schemes: recall that adversaries are allowed to query a signing key of a signer who will not follow instructions from the adversaries. In other words, the adversaries can obtain *read* access to the signer but not *write* access. One may argue that this assumption might be too complex to deploy the verifier and is out of touch with reality. To understand this issue, we can imagine the signer as a Non-Fungible Token (NFT) artist who will sign each of his/her artwork, generate a unique digital signature and trade them out for interest. Of course, this signer will store his/her artwork by some means, such as IPFS [14] or a hard disk. In this case, the signature has economic value and, therefore, is valuable to take the stateful signer into consideration. On the other hand, potential adversaries are also attracted by the economic value [15], which increases the necessity of this updatable notion. For any challenge epoch e^* and message m^* , we allow the adversary to query for the signatures of any message other than the m^* before epoch e^* . This is similar to basic existential unforgeability under adaptively chosen message attacks (EUF-CMA) security, where an adversary cannot submit a signature that has already been queried for as a challenge. However, in the USpt, a query to a signature in any epoch will cause it to be updated in all subsequent epochs because the update token is public. Therefore, we need to rule out this trivial attack.

1.4. Contribution

The contributions of this work can be concluded as follows:

1. define the model for updatable signature with public tokens;
2. prove two impossibilities of constructing secure USpt; and

3. construct a USpt based on BLS signature via aggregating property and prove its security.

The rest of the paper will be organized as follows. Section 2 will introduce some common terminology, define key homomorphic signatures, and discuss security concepts related to signatures. Section 3 will introduce updatable signatures and how we define security when the token is public. Section 4 discusses some construction impossibilities. After circumventing these impossibilities, we construct a USpt in Section 5 and prove its security in the model of Section 3. Finally, we conclude this paper in Section 6 and introduce future work.

2. Preliminaries

2.1. Notation

For $n \in \mathbb{N}$, let $[n] := \{1, \dots, n\}$, and let $\lambda \in \mathbb{N}$ be the security parameter. For a finite set $\mathcal{S}$, we denote by $s \leftarrow \mathcal{S}$ the process of sampling s uniformly from $\mathcal{S}$. Similarly, for an algorithm A , we denote by $y \leftarrow A(x)$ be the process of running A on input x with access to uniformly random coins and assigning the result to y . To make the random coins r explicit, we write $A(x; r)$. We use $\perp$ to indicate that an algorithm terminates with an error and A^B when A has access to call B , where B may return $\top$ as a distinguished special symbol. We say that an algorithm A runs in probabilistic polynomial time (PPT) if the running time of A is polynomial in security parameter λ . For some signature schemes, we denote the sets of signing keys, public keys, key-update tokens, and signature-update tokens by $\mathcal{SK}, \mathcal{PK}, \mathcal{KT}$, and $\mathcal{ST}$.

Let F be a binary relationship. We denote by $\mathcal{X}$ as the domain of F , $\mathcal{Y}$ as the range of F . We say F is a *function*, if $\forall x \in \mathcal{X}, \exists$ unique $y \in \mathcal{Y}$ s.t. xFy ; when we treat some elements x' of the domain as constants, we write the constants as the subscript $F_{x'}$. For function F , if xFy , we write it as $y = F(x)$. We say a function $F : \mathcal{X} \rightarrow \mathcal{Y}$, with domain X and range Y , is *bijective*, if $\forall y \in \mathcal{Y}, \exists$ unique $x \in \mathcal{X}$ s.t. $y = F(x)$. In this case, the *inverse function* of F is the function $F^{-1} : \mathcal{Y} \rightarrow \mathcal{X}$. Given two functions $g : \mathcal{X} \rightarrow \mathcal{Y}$ and $h : \mathcal{Y} \rightarrow \mathcal{Z}$, their *composition* is the function $h \circ g : \mathcal{X} \rightarrow \mathcal{Z}$ defined by $(h \circ g)(x) = h(g(x))$.

2.2. Key-Homomorphic Signature

Derler and Slamanig [16, 6] defined the key-homomorphic (KH) signature scheme, which is the main foundation of US with secret tokens [4]. We recall

relevant definitions here.

Definition 1 (Signature scheme). *A signature scheme Σ is a triple $(\text{KeyGen}, \text{Sign}, \text{Verify})$ of PPT algorithms, which are defined as follows:*

- $\text{Gen}(\lambda)$: *This algorithm takes a security parameter λ as input and outputs a secret (signing) key sk and a public (verification) key pk with associated message space $\mathcal{M}$.*
- $\text{Sig}(sk, m)$: *This algorithm takes a secret key sk and a message $m \in \mathcal{M}$ as input and outputs a signature σ .*
- $\text{Ver}(pk, m, \sigma)$: *This algorithm takes a public key pk , a message $m \in \mathcal{M}$ and a signature σ as input and outputs a verdict $b \in \{0, 1\}$.*

Definition 2 (Secret key to public key homomorphism). *A signature scheme Σ provides a secret key to public key homomorphism if there exists an efficiently computable map $\mu : \mathbb{H} \rightarrow \mathbb{E}$ such that for all $sk, sk' \in \mathbb{H}$ it holds that $\mu(sk + sk') = \mu(sk) \cdot \mu(sk')$, and for all $(sk, pk) \leftarrow \text{Gen}(\lambda)$, it holds that $pk = \mu(sk)$.*

Definition 3 (Key-homomorphic signatures). *A signature scheme is called key-homomorphic signature if it provides a secret key to public key homomorphism and an additional PPT algorithm Adapt , defined as:*

- $\text{Adapt}(pk, m, \sigma, \delta)$: *Given a public key pk , a message m , a signature σ , and a shift amount δ , this algorithm outputs a public key pk' and a signature σ' ,*

such that for all $\delta \in \mathbb{H}$ and all $(pk, sk) \leftarrow \text{Gen}(\lambda)$, all messages $m \in \mathcal{M}$ and all σ with $\text{Verify}(pk, m, \sigma) = 1$ and $(pk', \sigma') \leftarrow \text{Adapt}(pk, m, \sigma, \delta)$ it holds that

$$(\Pr[\text{Ver}(pk', m, \sigma') = 1] = 1) \wedge (pk' = \mu(\delta) \cdot pk).$$

Definition 4 (Perfect adaption). *A key-homomorphic signature scheme provides perfect adaption if for every $\lambda \in \mathbb{N}$, every message $m \in \mathcal{M}$, it holds that*

$$[\sigma, (sk, pk), \text{Adapt}(pk, m, \sigma, \delta)],$$

where $(sk, pk) \leftarrow \text{Gen}(\lambda)$, $\sigma \leftarrow \text{Sig}(sk, m)$, $\delta \leftarrow \mathbb{H}$, and

$$[\sigma, (sk, \mu(sk)), (\mu(sk) \cdot \mu(\delta), \text{Sig}(sk + \delta, m))],$$

where $sk \leftarrow \mathbb{H}$, $\sigma \leftarrow \text{Sig}(sk, m)$, $\delta \leftarrow \mathbb{H}$, are identically distributed.

Experiment $\text{Exp}_{\Sigma, \mathcal{A}}^{\text{euf-cma}}(\lambda)$:

$(pk, sk) \leftarrow \text{Gen}(\lambda)$
 $(m^*, \sigma^*) \leftarrow \mathcal{A}^{\text{sig}(sk, \cdot)}(pk, \lambda)$
 if $\text{sig}(sk, m^*)$ is not queried, return $\text{Ver}(pk, m^*, \sigma^*)$; otherwise, re-
 turn 0

Figure 1: The EUF-CMA security experiment.

US from [4] is mainly built on the algorithm **Adapt**, and the perfect adaption ensures the unlinkable updates (Definition 10) and indistinguishable simulation in security proof.

The most basic security requirement for a signature scheme is universal unforgeability under no-message attacks (UUF-NMA).

Definition 5 (UUF-NMA). *A signature scheme Σ is UUF-NMA secure iff for all PPT adversary $\mathcal{A}$ such that*

$$\Pr[(sk, pk) \leftarrow \text{Gen}(\lambda), m^* \leftarrow \mathcal{M}, \sigma^* \leftarrow \mathcal{A}(pk, m^*, \lambda) : \text{Ver}(pk, m^*, \sigma^*) = 1]$$

is negligible in λ .

We also require EUF-CMA security.

Definition 6 (EUF-CMA). *A signature scheme Σ is EUF-CMA secure iff for any valid PPT adversary $\mathcal{A}$ such that*

$$\text{Adv}_{\Sigma, \mathcal{A}}^{\text{euf-cma}}(\lambda) := \Pr[\text{Exp}_{\Sigma, \mathcal{A}}^{\text{euf-cma}}(\lambda) = 1]$$

is negligible in security parameter λ where $\text{Exp}_{\Sigma, \mathcal{A}}^{\text{euf-cma}}(\lambda) = 1$ is defined in Figure 1.

3. Updatable Signature with Public Tokens

This section defines USpt and its security model.

3.1. Defining the Updatable Signature with Public Tokens

First, let us review the definition of US with secret tokens in [4].

Definition 7 (Updatable Signature). *An US scheme US is a tuple of the five PPT algorithms (Setup, Next, Sig, Update, Ver):*

- **Setup**(λ, n): On input security parameter λ and the maximum number of epochs $n \in O(2^\lambda)$, the setup algorithm outputs a public and secret key pair (pk_1, sk_1) .
- **Next**(pk_e, sk_e): On input a public key pk_e and signing key sk_e for epoch $e \in [n - 1]$, the key-update algorithm outputs an updated public key pk_{e+1} , an updated signing key sk_{e+1} , and a update token Δ_{e+1} .
- **Sig**(sk_e, m): On input signing key sk_e for epoch $e \in [n]$ and a message $m \in \mathcal{M}$, the signing algorithm outputs a signature σ_e .
- **UpdateS**($\Delta_{e+1}, m, \sigma_e$): On input an update token Δ_{e+1} , a message m , and a signature σ_e for epoch $e < n$, the update algorithm outputs an updated message-signature pair (m, σ_{e+1}) or $\perp$.
- **Ver**(pk_e, m, σ_e): On input public key pk_e , a message m , and a signature σ_e for epoch $e \in [n]$, the verification algorithm outputs a verdict $b \in \{0, 1\}$.

In the US, we have two kinds of signatures: the signature generated by **Sig** algorithm and the signatures generated by **UpdateS** algorithm. We refer to the former signatures (generated by **Sig**) as *fresh* signatures and the latter signatures (generated by **UpdateS**) as *updated* signatures.

Now, let us introduce the USpt. Intuitively, we want to update a signing key sk_e to an updated key sk_{e+1} first, then update signatures under outdated key σ_e to new signatures under the updated key σ_{e+1} . Since we are trying to update different subjects in the above-mentioned two steps, we define two kinds of tokens:

- δ : key-update token;
- Δ : signature-update token.

We model the signed message database as a set of messages denoted by M . The key-update token δ is generated by algorithm **Next** and used firstly as an element to compute the next signing key. Then, the algorithm **Next** takes δ and M as input to compute a signature-update token Δ . In our definition, the key-update token δ is only used as an *intermediate* for computing the

next public-secret key pair. We also need to add a verification step in the update signature algorithm **UpdateS** for limiting the signature-update token only to be used to update signed messages in set M .

Definition 8 (Updatable Signature with Public Tokens). *An USpt scheme USpt is a tuple of the PPT algorithms (Setup, Next, Sig, UpdateS, Ver):*

- **Setup**(λ, n): On input security parameter λ and the maximum number of epochs $n \in O(2^\lambda)$, the setup algorithm outputs a public and secret key pair (pk_1, sk_1) and an initially empty signed-message set M .
- **Next**(pk_e, sk_e, M): On input a public key pk_e , signing key sk_e for epoch $e \in [n - 1]$, and a signed-message set M , the next-key algorithm computes an updated public key pk_{e+1} , an updated secret key sk_{e+1} , and a signature-update token Δ_{e+1} .
- **Sig**(sk_e, m): On input signing key sk_e for epoch $e \in [n]$ and a message $m \in \mathcal{M}$, the signing algorithm outputs a signature σ_e .
- **UpdateS**($\Delta_{e+1}, m, \sigma_e$): On input a signature-update token Δ_{e+1} , a message m , and a signature σ_e for epoch $e < n$, the update algorithm outputs a message and updated message-signature pair (m, σ_{e+1}) or $\perp$.
- **Ver**(pk_e, m, σ_e): On input public key pk_e , a message m , and a signature σ_e for epoch $e \in [n]$, the verification algorithm outputs a verdict $b \in \{0, 1\}$.

3.1.1. Correctness of USpt

For all $\lambda, n \in \mathbb{N}$, for all $(pk_1, sk_1) \leftarrow \text{Setup}(\lambda, n)$, for all $e \in [n - 1]$, for all $(pk_{e+1}, sk_{e+1}, \Delta_{e+1}) \leftarrow \text{Next}(pk_e, sk_e, M)$, for all σ_e with $\text{Ver}(pk_e, m, \sigma_e) = 1$, for all $(m, \sigma_{e+1}) \leftarrow \text{UpdateS}(\Delta_{e+1}, m, \sigma_e)$, we have that $\Pr[\text{Ver}(pk_{e'}, m, \sigma_{e'}) \neq 1] \leq \epsilon(\lambda)$ holds, for all $e' \in [n]$, where $\epsilon(\lambda) = \text{negl}(\lambda)$, and we call it perfectly correct if $\epsilon(\lambda) = 0$.

3.2. Security Model

3.2.1. Global State

In our security game/experiments, let $q \in \mathbb{N}$ be the number of signature queries and e be the identifier of the current epoch. Next, we define a global state $\mathbf{S} = (\mathcal{I}, \mathcal{K}, \mathcal{S})$:

- $\mathcal{I} = \{((pk_{e'}, sk_{e'}), \Delta_{e'})_{e' \in [e]}\}$: all keys and signature-update tokens;
- $\mathcal{K} = \{e' \in [e]\}$: all epochs where the adversary queried **corrupt**(e');

- $\mathcal{S} = \{(e', m, \sigma_{e'})_{e' \in [e]}\}$: all tuples where the adversary queried $\text{sig}(m, e')$ in epoch e' or $\text{updates}(\Delta_{e'}, m, \cdot)$ in epoch $e' - 1$.

The set $\mathcal{I}$ is initialized by $\{((pk_1, sk_1), \Delta_1)\}$, where $(pk_1, sk_1) \leftarrow \text{Setup}(\lambda, n)$ and $\Delta_1 := \perp$, and other three sets $\mathcal{K}$, $\mathcal{T}$ and $\mathcal{S}$ are initialized as empty.

Set $\mathcal{I}$ is for recording the key pairs and key-update tokens; set $\mathcal{K}$ is for recording the epochs that the signing keys of which have been obtained by the adversary; set $\mathcal{S}$ is also used for recording the adversarial signature requests, e.g., logging the signatures queried by the adversary and corresponding epochs and messages. Unlike the US security model [4], we do not need to record the intermediate key-update token δ since it is not helpful for future signing and also key updates. In other words, δ is not used as an intermediate value and is not transmitted via any channels; also, our signature-update token is publicly queriable.

With access to the global state $\mathbf{S} = (\mathcal{I}, \mathcal{K}, \mathcal{S})$, for any epoch $e' \in [e]$ the adversary is also given the following oracles (sig , next , updates , corrupt , ver):

$\text{sig}(m, e')$: On input message m and epoch e' , compute signature $\sigma_{e'} \leftarrow \text{Sig}(sk_{e'}, m)$, update $\mathcal{S} := \mathcal{S} \cup \{(e', m, \sigma_{e'})\}$, and return $\sigma_{e'}$.

next : Find $(pk_e, sk_e) \in \mathcal{I}$, compute $(pk_{e+1}, sk_{e+1}, \Delta_{e+1}) \leftarrow \text{Next}(pk_e, sk_e)$, update $\mathcal{I} := \mathcal{I} \cup \{((pk_{e+1}, sk_{e+1}), \Delta_{e+1})\}$, return (pk_{e+1}, Δ_{e+1}) and set $e := e + 1$.

$\text{updates}(\Delta_{e'+1}, m, \sigma_{e'})$: On input a token-signature pair $(\Delta_{e'+1}, \sigma_{e'})$, return $\perp$ if $\text{ver}(m, \sigma_{e'}) \neq 1$; otherwise, compute $t \leftarrow \text{UpdateS}(\Delta_{e'+1}, m, \sigma_{e'})$, return $\perp$ if $t = \perp$; otherwise, parse $t = (m, \sigma_{e'+1})$, update $\mathcal{S} := \mathcal{S} \cup \{(e' + 1, m, \sigma_{e'+1})\}$ and return $(m, \sigma_{e'+1})$.

$\text{corrupt}(e')$: On input an epoch $e' \in [e]$, return $\perp$ if $e' \geq e$; otherwise, return $sk_{e'}$ and update $\mathcal{K} := \mathcal{K} \cup \{e'\}$.

$\text{ver}(m, \sigma_{e'})$: On input a message-signature pair $(m, \sigma_{e'})$, return $b := \text{Ver}(pk_{e'}, m, \sigma_{e'})$.

3.2.2. Leakage Profile

Considering the relevant signatures between epochs, different from [4], our signature-update token is public; therefore, any valid signature in any previous epoch can be updated to the current activated epoch if one logs all the signature-update tokens from the previous epoch to now. In this case, we only need to consider whether the adversary can forge a different signature-message pair when given all valid message-signature pairs obtained from sig .

Experiment $\text{Exp}_{\text{USpt}, \mathcal{A}}^{\text{uspt-euf-cma}}$:

$(pk_1, sk_1) \leftarrow \text{Setup}(\lambda, n)$

$\mathbf{S} = (\mathcal{I}, \mathcal{K}, \mathcal{S})$, for $\mathcal{I} := \{((pk_1, sk_1), \perp)\}$, $\mathcal{K} := \mathcal{S} := \emptyset$

$(m^*, \sigma_{e^*}^*, e^*) \leftarrow \mathcal{A}^{\text{sig, next, updates, corrupt, ver}}(\lambda)$

if for some $e' \leq e^*$, $(e', m^*, \cdot) \in \mathcal{S}$ or $e^* \in \mathcal{K}$, return 0; otherwise,

return $\text{Ver}(pk_{e^*}, m^*, \delta_{e^*}) = 1$

Figure 2: USpt-EUF-CMA security experiment.

This requirement is similar to the basic EUF-CMA security of plain signature schemes.

However, we should also restrict the *unchallengeable* signatures when the signing key is leaked. Ideally, one signing key leakage should only affect the security in that epoch, and that is the reason we need such an updatable property. Therefore, the global state $\mathcal{K}$ and $\mathcal{S}$ is sufficient for recording all adversarial operations considering public tokens. This limitation can be presented in the following security definition.

3.2.3. Existential Unforgeability Under Chosen-Message Attacks (USpt-EUF-CMA)

Informally, the USpt-EUF-CMA captures the security notions that no PPT adversary can forge a valid signature, even given the compromising oracles. We say that a USpt scheme USpt is USpt-EUF-CMA-secure if no PPT adversary $\mathcal{A}$ succeeds in the USpt-EUF-CMA security experiment (Fig. 2) with non-negligible probability.

To keep the adversary valid, it cannot query **sig** on the challenge message m^* on any epoch before e^* since the signature-update tokens are public, any signature in an epoch earlier than e^* can be updated to e^* by any adversary. On the contrary, signatures in an epoch greater than e^* cannot be updated to e^* . In addition, the signing key has to be kept secret in epoch e^* .

Definition 9 (USpt-EUF-CMA security). *A USpt scheme USpt is USpt-EUF-CMA-secure iff for any PPT adversary $\mathcal{A}$ the advantage function*

$$\text{Adv}_{\text{USpt}, \mathcal{A}}^{\text{uspt-euf-cma}}(\lambda, n) := \Pr \left[\text{Exp}_{\text{USpt}, \mathcal{A}}^{\text{uspt-euf-cma}}(\lambda, n) = 1 \right],$$

is negligible in λ , where $\text{Exp}_{\text{USpt}, \mathcal{A}}^{\text{uspt-euf-cma}}(\lambda, n)$ is defined in Fig .2.

Experiment $\text{Exp}_{\text{USpt}, \mathcal{A}}^{\text{uspt-uu-cma}}$:

$(pk_1, sk_1) \leftarrow \text{Setup}(\lambda, n)$

$\mathbf{S} = (\mathcal{I}, \mathcal{K}, \mathcal{S})$, for $\mathcal{I} := \{((pk_1, sk_1), \perp)\}$, $\mathcal{K} := \mathcal{S} := \emptyset$

$(m^*, e', e^*) \leftarrow \mathcal{A}^{\text{sig, next, updates, corrupt, ver}}(\lambda)$

$b \leftarrow \{0, 1\}$

$\sigma^0 \leftarrow \text{Sig}(sk_e^*, m^*)$, $\sigma^1 \leftarrow \text{UpdateSch}(m^*, e', e^*)^a$

$b^* \leftarrow \mathcal{A}^{\text{sig, next, updates, corrupt, ver}}(\lambda, \sigma^b)$

if $(e', m^*, \cdot) \in \mathcal{S}$, $e' < e^*$, and $b^* = b$, return 1; otherwise, return 0

^aWe use a compact notation $\text{UpdateSch}(m^*, e', e^*)$ to denote the repeated application of `updates` starting from epoch e' to forward the signature $\sigma_{e'}$ to the final signature σ_{e^*} in epoch e^* , then we assign σ_{e^*} to σ^1

Figure 3: USpt-UU-CMA security experiment.

3.2.4. Unlinkable Updates Under Chosen-Message Attacks (USpt-UU-CMA)

Informally, USpt-UU-CMA captures the security notions that no PPT adversary can distinguish fresh signatures from updated signatures, even given all signing keys, key/signature-update tokens, and signatures from past epochs. We say that an USpt scheme USpt is USpt-UU-CMA-secure if any PPT adversary $\mathcal{A}$ succeeds in the USpt-UU-CMA security experiment (Fig. 3) with negligible advantage.

Definition 10 (USpt-UU-CMA security). *A USpt scheme USpt is USpt-UU-CMA-secure iff for any valid PPT adversary $\mathcal{A}$ the advantage function*

$$\text{Adv}_{\text{USpt}, \mathcal{A}}^{\text{uspt-uu-cma}}(\lambda, n) := \left| \Pr \left[\text{Exp}_{\text{USpt}, \mathcal{A}}^{\text{uspt-uu-cma}}(\lambda, n) = 1 - \frac{1}{2} \right] \right|,$$

is negligible in λ , where $\text{Exp}_{\text{USpt}, \mathcal{A}}^{\text{uspt-uu-cma}}(\lambda, n)$ is defined in Fig. 3.

4. Signature-update Token and Key-update Token

In this section, we will show why USpt cannot be trivially constructed from the Key-Homomorphic signature-based US (KH-based US).

4.1. Decoupling Update Tokens

Let Σ be a KH signature, the KH-based US is shown in Figure 4. To clarify the difference, we define $\text{UpdateK}(\delta_{e+1}, sk_e) = sk_e + \delta_{e+1}$, which is called as a subroutine in `Next`.

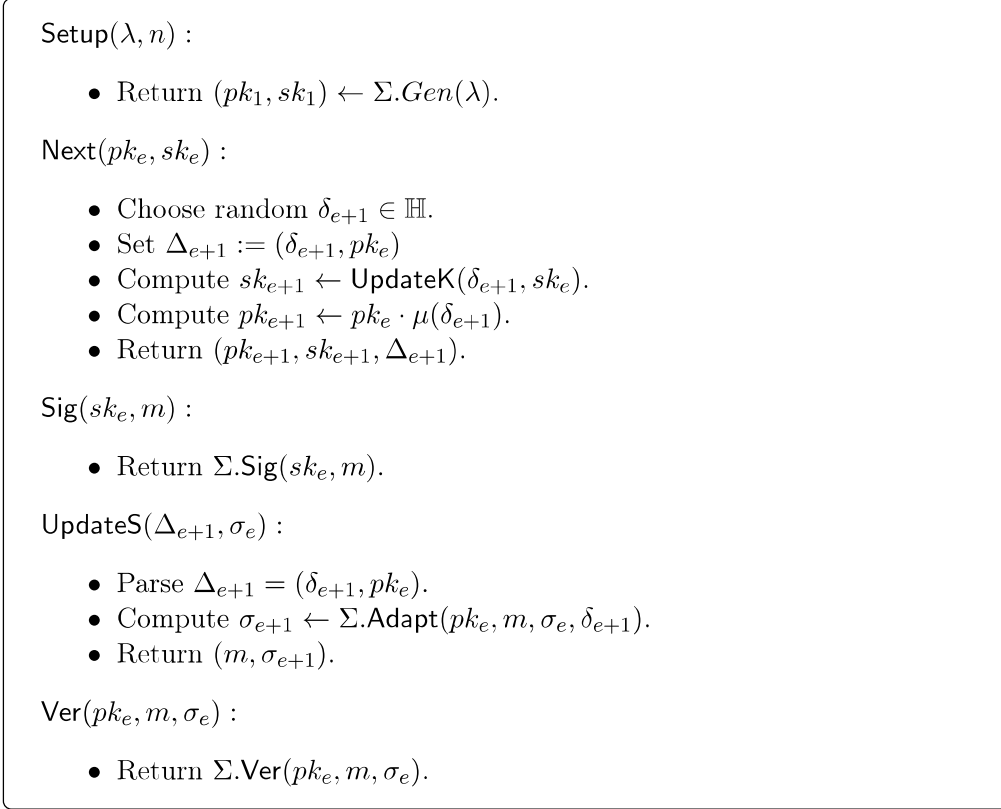


Figure 4: KH-based US

We found that the **Next** algorithm first updates the key pair by the key-update token δ_{e+1} , and then outputs the signature-update token Δ_{e+1} which is a tuple including δ_{e+1} . This fact induces a signing key sk_e to be updated by the signature-update token Δ_{e+1} easily. That is, one can compute a *transform function* f such that $f(\Delta_{e+1}) = \delta_{e+1}$. Therefore, no KH-based US is secure by trivially publicizing the signature-update tokens since the adversary can use it to update the signing key and continually forge signatures. We also found this problem is inherent in the KH-based US since all those updatabilities are achieved by the **Adapt** algorithm (Definition 3), and the security of KH-based US relies on the perfect adaptability (Definition 4). Therefore, we should avoid computing key-update tokens from the signature-update tokens. There are two natural ways to achieve this goal: constructing a USpt without f or constructing a USpt with infeasible computable f . The latter one is a bit

more complex; one can understand that f is the inverse of a one-way function f^{-1} . Next, we prove that the first idea is not achievable when the fresh signatures and the updated signatures are identical, and the updatability in the second solution can not be achieved via the perfect adaptability notion.

4.2. Existence of Transform Function f

Note that the perfect adaptability implies the signature obtained from **Adapt** is identical to the signature obtained from **Sig** given the same message m , key pair (sk_e, pk_e) , and tokens $(\delta_{e+1}, \Delta_{e+1})$. We define a notion for US schemes to unveil the equality of fresh and updated signatures (EFUS).

Definition 11 (Equality of Fresh and Updated Signatures). *A US scheme US is EFUS, if for any epoch $e, e+1 \in [n]$, signing key $sk_e, sk_{e+1} \in \mathcal{SK}$, tokens $\Delta_{e+1} \in \mathcal{ST}, \delta_{e+1} \in \mathcal{KT}$, $r \in \mathcal{R}$, and $m \in \mathcal{M}$, where $\mathcal{SK}, \mathcal{ST}, \mathcal{KT}, \mathcal{R}$ and $\mathcal{M}$ are the sets of signing keys, signature-update tokens, key-update tokens, randomness and messages, such that*

$$\text{Sig}(\text{UpdateK}(\delta_{e+1}, sk_e), m; r) = \text{UpdateS}(\Delta_{e+1}, m, \text{Sig}(sk_e, m; r)). \quad (1)$$

If the signature scheme is not a unique signature scheme, we obtain different signatures on the same message using different randomness with overwhelming probability. That is why we explicitly define the randomness in the above definition. Now, we are ready to prove the impossibility of constructing an EFUS-US without f .

Lemma 1. *Suppose a US scheme US is EFUS. Given any $m \in \mathcal{M}, r \in \mathcal{R}, sk_e \in \mathcal{SK}$, we can rewrite the left side of (1) as*

$$\text{Sig}_{m,r}(\text{UpdateK}_{sk_e}(\delta_{e+1}));$$

then, function f exists, if functions $\text{Sig}_{m,r}$ and UpdateK_{sk_e} are bijective.

Proof. Because $\text{Sig}_{m,r}$ and UpdateK_{sk_e} are bijective, therefore they are invertible, and their composite function $g := \text{Sig}_{m,r} \circ \text{UpdateK}_{sk_e}$ is also invertible, the inverse of which is denoted as $g^{-1} : \mathcal{S} \rightarrow \mathcal{KT}$. We rewrite the right side of (1) as $\text{UpdateS}_{m,r,sk_e}(\Delta_{e+1})$, and denote it as $h(\Delta_{e+1})$ for short, where $h : \mathcal{ST} \rightarrow \mathcal{S}$ and we hide $\text{Sig}(sk_e, m; r)$ because after fixing m, r, sk_e , it is also a constant. Therefore, we can construct $f(\Delta_{e+1}) = g^{-1}(h(\Delta_{e+1}))$, where $f : \mathcal{ST} \rightarrow \mathcal{S} \rightarrow \mathcal{KT}$. Lemma 1 is proved. $\square$

Note that Lemma 1 applies to all eligible US that are in line with the assumption, not only KH-based US. However, it is not necessary to construct f according to the steps in the proof, which may introduce discrete logarithm problems. We show the fact that the transform function f exists in BLS-based US with message-independent updates here.

4.2.1. Transform Function f in BLS-based US with Message-independent Updates

For BLS-based US, the main difference from KH-based US is that the update of the signing key uses multiplication instead of addition in the `UpdateK` algorithm.

The message-independent updates property allows the signature-update algorithm to be run without inputting messages. We modify the `Next` algorithm from [4] a little bit, from computing multiplication $sk_{e+1} \leftarrow sk_e \cdot \delta_{e+1}$ to computing modular multiplication $sk_{e+1} \leftarrow sk_e \cdot \delta_{e+1} \bmod p$, for signing key size control and succinct proof.

One can easily verify that BLS-based US with message-independent updates is EFUS.

Corollary 1. *In BLS-US with message-independent updates (Fig. 5), functions Sig_m and UpdateK_{sk} are bijective.*

Proof. *We prove it by contradiction.*

According to the definition, the function $\text{UpdateK}_{sk}(\delta) := sk \cdot \delta \bmod p$, where $sk, \delta \leftarrow \mathbb{Z}_p^*$, which is typically a prime-order modular multiplication, obviously bijective.

Next, we prove that Sig_m is injective. Given a message m (since there is no randomness used in $\text{Sig}_{m,r}$, we omit r for clarity), suppose there are two different signing keys $sk_1 \neq sk_2$, such that $\text{Sig}_m(sk_1) = \text{Sig}_m(sk_2)$, i.e., $H(m)^{sk_1} = H(m)^{sk_2}$. Since $H : \mathcal{M} \rightarrow \mathbb{G}_1$, and every element in $\mathbb{G}_1$ can be represented by g , say $H(m) = g^t$, then,

$$H(m)^{sk_1} = H(m)^{sk_2} \Rightarrow g^{t \cdot sk_1} = g^{t \cdot sk_2} \Rightarrow g^{(t \cdot sk_1) - (t \cdot sk_2)} = 1.$$

Therefore, $(t \cdot sk_1) - (t \cdot sk_2) = 0$, $sk_1 = sk_2$, which contradicts our assumption (note that the computation is not over the cyclic group).

Now we prove that Sig_m is surjective. Informally, for every element in the range of Sig_m , there is always at least one element in the domain corresponding to it. We can derive its surjective property of Sig_m from the correctness

Setup(λ, n) :

- Choose three groups $\mathbb{G}_1 = \langle g \rangle$, $\mathbb{G}_2 = \langle \tilde{g} \rangle$, and $\mathbb{G}_T$ are three groups of prime order p with $\lambda = \log_2 p$; construct a bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$; choose a hash function $H : \mathcal{M} \rightarrow \mathbb{G}_1$ uniformly at random from hash function family $\{H_k\}_k$.
- Choose $sk \leftarrow \mathbb{Z}_p^*$, and set $pk := \tilde{g}^{sk}$.
- Return $(pk_1, sk_1) \leftarrow (pk, sk)$.

Next(pk_e, sk_e) :

- Choose $\delta_{e+1} \leftarrow \mathbb{Z}_p^*$, and set $\Delta_{e+1} := (\delta_{e+1}, pk_e)$.
- Compute $pk_{e+1} \leftarrow pk_e^{\delta_{e+1}}$.
- UpdateK(δ_{e+1}, sk_e) : Compute $sk_{e+1} \leftarrow sk_e \cdot \delta_{e+1} \pmod p$.
- Return $(pk_{e+1}, sk_{e+1}, \Delta_{e+1})$.

Sig(sk_e, m) :

- Return $\sigma_e := H(m)^{sk_e}$.

UpdateS(Δ_{e+1}, σ_e) :

- Parse $\Delta_{e+1} = (\delta_{e+1}, pk_e)$.
- Compute $\sigma_{e+1} \leftarrow \sigma_e^{\delta_{e+1}}$.
- Return σ_{e+1} .

Ver(pk_e, m, σ_e) :

- Return $e(H(m), pk_e) = e(\sigma_e, \tilde{g})$.

Figure 5: BLS-based US with message-independent updates

of the BLS signature: for every valid signature, there must be a valid public key verifying it; and in addition, every valid public key has a corresponding signing key; therefore, the surjective property of Sig_m follows. $\square$

With the above proof, we can construct the transform function f :

$$f := \text{UpdateK}_{sk_e}^{-1} \circ \text{Sig}_m^{-1} \circ \text{UpdateS}_{sk_e}.$$

Given a signature $\sigma_e = H(m)^{sk_e}$, we can update it to $H(m)^{sk_e \cdot \delta_{e+1}}$ by inputting signature-update token $\Delta_{e+1} = (\delta_{e+1}, pk_e)$ to UpdateS_{sk_e} . Then we

can apply Sig_m^{-1} to get $sk_{e+1} = sk_e \cdot \delta_{e+1}$ (which involves the discrete logarithm problems on group $\mathbb{G}_1$). Finally, one can use $\text{UpdateK}_{sk_e}^{-1}(sk_{e+1})$ to get the key-update token δ_{e+1} . Note that Lemma 1 only implies the existence of f , not promising a “feasible” construction. Indeed, computing the signing key from a signature is always a devastating task for the security of a signature scheme.

This fact interestingly implies the possible way to construct a computationally infeasible function f , e.g., making it hard to compute (non-polynomially time computable) the key-update token from a signature-update token or, making f^{-1} *one-way*.

4.3. One-wayness of Transform Function f^{-1}

A straightforward way is to publish the public-key version of the key-update token. As explained in Lemma 1 of [6], if there is a KH signature scheme such that we can replace the update token δ in the **Adapt** algorithm (Definition 3) with its public key $\mu(\delta)$, this scheme will not provide UUF-NMA (5) since any adversary can update a signature signed by signing key sk to a forged signature that can be verified by the public key $pk^* = \mu(sk) \cdot \mu(\delta)$ without knowledge of δ .

A key point in the informal proof above is that one can compute the signature-update token from any valid signing key and the updated (next epoch’s) public key, e.g., $\mu(\delta) \leftarrow pk^* \cdot \mu(sk)^{-1}$. This relationship is intrinsic for the KH-based US.

We claim this lemma in a more *general* version, where if there is a US scheme with *any public* signature-update token Δ (in the case of KH-based US, $\Delta := \mu(\delta)$) that can update all signatures from one epoch to the next, then the signature scheme is not UUF-NMA secure. This corollary sufficiently excludes the possibility of building USpt from perfect adaptability.

Corollary 2. *A US scheme US for which its PPT algorithm Update takes a signature-update token $\Delta_{e+1} \in \mathcal{PK}$ that still satisfies Definition 7 and there exists an efficiently computable function $g(pk_e, pk_{e+1}) \rightarrow \Delta_{e+1}$, then US is not UUF-NMA-secure.*

Proof. We prove this corollary by constructing an adversary against UUF-NMA security of US. Let $\mathcal{C}$ be an UUF-NMA challenger with input pk_{e+1} and a target message m^* , its goal is to output a valid signature σ_{e+1} such that $\text{Ver}(pk_{e+1}, m, \sigma_{e+1}) = 1$. $\mathcal{C}$ runs **Setup**(λ, n) to get a key pair (pk_1, sk_1)

and sets $(pk_e, sk_e) := (pk_1, sk_1)$, then computes $\sigma_e \leftarrow \text{Sig}(sk_e, m)$, and finally, outputs a forgery $\sigma_{e+1} \leftarrow \text{UpdateS}(g(pk_e, pk_{e+1}), m, \sigma_e)$. The success of forgery follows the correctness of updatable signatures. $\square$

An implicit condition is that each signature-update token Δ_{e+1} can update *all* signatures generated in epoch e to epoch $e + 1$. In addition, the existence and bijective of f implies that each signature-update token is bonded to each update of epochs. We need to break this paradigm for other possible USpt constructions. Therefore, if the token is only useful for updating the message signed by the signer, then this kind of attack will fail. The problem is to publish a token that makes the verifiers distinguish the signature signed by the signer from the signature signed by the adversary, which is similar to the problem of proving if some *elements* (signatures) are in a *set* (signature signed by the signer). We will discuss this issue in detail below.

5. Construction of USpt from the BLS signature

This section introduces our solution for constructing a USpt from BLS.

5.1. USpt from BLS Signatures

The USpt scheme BLS-USpt is shown in Fig. 6. The signed message database for the signer can be more efficiently constructed by replacing the input message m to the hash of it, $H(m)$, therefore saving the storage of the signer. This method of publishing signature-token will not affect the security of key-token since deriving key-token δ_{e+1} from $H(m)^{\delta_{e+1}}$ is solving an instance of discrete logarithm problem in group $\mathbb{G}_1$. In this case, the signer who wants to update the signing keys and keep the validity of previously generated signatures must record hashes of the messages signed before. For every signature update, the signer needs to add an element $H(m)^{\delta_{e+1}}$ to the signature-update token Δ_{e+1} . We deem this difference between the existing US schemes [4] as a new scenario application. Indeed, considering a trusted agent to update every signature on behalf of the signer, the one-for-all token is much easier to use and reduces the bandwidth during the process. One-for-one tokens still enable efficient updates without a trusted third party.

This method is surprisingly related to the notion of signature aggregation [17, 18]. In the scenario of signature aggregation, given n signatures $\sigma = (\sigma_1, \sigma_2, \dots, \sigma_n)$ for message $m = (m_1, m_2, \dots, m_n)$ from n signers with public key $pk = (pk_1, pk_2, \dots, pk_n)$, the verifier is able to verify those n signatures together in just one call to aggregate verification algorithm **AVer**:

$\text{AVer}(\sigma, pk, m) :$

- Compute $\sigma' = \prod_{i=1}^n \delta_i$.
- Compute $h_i \leftarrow H(m_i)$ for all $1 \leq i \leq n$.
- Return $\prod_{i=1}^n e(h_i, pk_i) = e(\sigma', \tilde{g})$.

In the case of updatable signatures, suppose the signer wants to update a signature $\sigma_e = H(m)^{sk_e}$ held by a verifier; the signer sends to the verifier a signature-update token, which contains a corresponding update token $(H(m)^{\delta_{e+1}}, pk_{e+1})$; then the verifier updates the σ_e to $\sigma_{e+1} = \sigma_e \cdot H(m)^{\delta_{e+1}}$. Note that $H(m)^{\delta_{e+1}}$ is actually a signature under signing key δ_{e+1} . Finally, the verification algorithm is the same as single signature verification: return $e(H(m), pk_{e+1}) = e(\sigma_{e+1}, \tilde{g})$. This verifies a message m by aggregating two different public keys $pk_{e+1} = pk_e \cdot \tilde{g}^{\delta_e}$.

A basic requirement of BLS aggregation is that messages in that aggregation should be distinct. Regarding updatable signatures, the verifier can verify signatures on the same message in different epochs. This is not a problem since we assume the update tokens are authenticated. We will show the details in the formal proof.

5.2. Security of BLS-USpt

We first prove that BLS-USpt is USpt-UU-CMA (Definition 10) secure, then USpt-EUF-CMA (Definition 9) secure.

5.2.1. USpt-UU-CMA

Formally, the adversary $\mathcal{A}$ is given `sig`, `next`, `updates`, `corrupt`, and `ver` oracles, and then it returns a target epoch e^* and a message m^* which it queried $\sigma_{e'}^* \leftarrow \text{sig}(m^*)$ in some epoch $e' < e^*$. In game 0, the experiment $\text{Exp}_{\text{BLS-USpt}, \mathcal{A}}^{\text{uspt-euf-cma}}$ with $b = 0$ returns $\sigma^{(0)} \leftarrow \text{Sig}(sk_{e^*}, m^*)$ to $\mathcal{A}$. In game 1, the experiment $\text{Exp}_{\text{BLS-USpt}, \mathcal{A}}^{\text{uspt-euf-cma}}$ with $b = 1$ returns $\sigma^{(1)} \leftarrow \text{UpdateSch}(m^*, e', e^*)$ to $\mathcal{A}$. Note that `UpdateSch` calls `Next` to get the signature-update tokens as input, then the adversary $\mathcal{A}$ also accesses the tokens $(\Delta_{e'+1}, \Delta_{e'+2}, \dots, \Delta_{e^*})$. Afterwards, $\mathcal{A}$ returns 0 if `updates` $(\Delta_{e'+1}, m^*, \sigma_{e'}^*) = 0$ and 1 otherwise.

Theorem 1. *The BLS-USpt from Figure 6 is USpt-UU-CMA secure.*

Proof. *Formally, we consider the following sequence of games:*

Game 0. *This is the experiment $\text{Exp}_{\text{BLS-USpt}, \mathcal{A}}^{\text{uspt-euf-cma}}$ with $b = 0$, where we return $\sigma^{(0)} \leftarrow \text{Sig}(sk_{e^*}, m^*)$.*

Setup(λ, n) :

- Choose three groups $\mathbb{G}_1 = \langle g \rangle$, $\mathbb{G}_2 = \langle \tilde{g} \rangle$, and $\mathbb{G}_T$ are three groups of prime order p with $\lambda = \log_2 p$; construct a bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$; choose a hash function $H : \mathcal{M} \rightarrow \mathbb{G}_1$ uniformly at random from hash function family $\{H_k\}_k$; set an empty signed-message-hash set $\mathcal{H} := \emptyset$.
- Choose $sk \leftarrow \mathbb{Z}_p^*$, and set $pk := \tilde{g}^{sk}$.
- Set $(pk_1, sk_1) := (pk, sk)$.
- Return $(pk_1, sk_1, \mathcal{H})$.

Next($pk_e, sk_e, \mathcal{H}$) :

- Choose $\delta_{e+1} \leftarrow \mathbb{Z}_p^*$.
- UpdateK(δ_{e+1}, sk_e) : Compute $sk_{e+1} \leftarrow sk_e + \delta_{e+1} \pmod p$.
- Compute $pk_{e+1} \leftarrow \tilde{g}^{sk_{e+1}}$.
- Compute $h_i^{\delta_{e+1}}$ for all $h_i \in \mathcal{H}$.
- Set signature-update token $\Delta_{e+1} := (h_i, h_i^{\delta_{e+1}})_{h_i \in \mathcal{H}} \cup pk_{e+1}$.
- Return $(pk_{e+1}, sk_{e+1}, \Delta_{e+1})$.

Sig(sk_e, m) :

- Set $\mathcal{H} := \mathcal{H} \cup H(m)$.
- Return $\sigma_e \leftarrow H(m)^{sk_e}$.

UpdateS($\Delta_{e+1}, \sigma_e, m$) :

- Find $(H(m), \Delta_{H(m), e+1}) \in \Delta_{e+1}$ using $H(m)$; if not found, return 0.
- Return $\sigma_{e+1} \leftarrow \sigma_e \cdot \Delta_{H(m), e+1}$.

Ver(pk_e, m, σ_e) :

- Return $e(H(m), pk_e) = e(\sigma_e, \tilde{g})$.

Figure 6: BLS-USpt

Game 1. This is the experiment $\text{Exp}_{\text{BLS-USpt}, \mathcal{A}}^{\text{uspt-euf-cma}}$ with $b = 1$, where we return $\sigma^{(1)} \leftarrow \text{UpdateSch}(m^*, e', e^*)$.

Note that the adversary $\mathcal{A}$ is given sig, next, updates, corrupt, and ver oracles, and therefore access to all keys, tokens, and signatures. In **Game 0**, let the key pair in epoch e^* be (sk_{e^*}, pk_{e^*}) . In **Game 1**, let the key pair

in epoch e' be $(sk_{e'}, pk_{e'})$. We define (sk_{e^*}, pk_{e^*}) for **Game 1** by **Next** and record all keys and signature-update tokens in the global state set $\mathcal{I}$. Then, algorithm $\text{UpdateSch}(m^*, e', e^*)$: works as follows: for $i = e', \dots, e^* - 1$, compute $\sigma_{i+1} \leftarrow \text{UpdateS}(\Delta_{i+1}, m^*, \sigma_i)$, where $\sigma_{i+1} = \sigma_i \cdot \Delta_{H(m^*), i+1}$; return σ_{e^*} .

Since BLS-USpt is EFUS, therefore the output of UpdateSch $\sigma_{e^*} = \sigma^{(1)}$ is identical to the signature $\sigma^{(0)}$. This concludes that $\text{Adv}_{\text{BLS-USpt}, \mathcal{A}}^{\text{uspt-uu-cma}}(\lambda, n) = 0$ for any adversary $\mathcal{A}$, which concludes the proof of USpt-UU-CMA security. $\square$

5.2.2. USpt-EUF-CMA

We reduce USpt-EUF-CMA security of BLS-USpt to the EUF-CMA security of BLS¹. Specifically, we use the **Sig** oracle of EUF-CMA challenger in epoch e^* . Except for this challenge epoch, we have to answer **sig**, **updates**, and **corrupt** queries, where the last one will leak the signing key to the adversary. Recall that a valid challenge should be formed like $(m^*, \sigma_{e^*}^*, e^*)$, where no **sig** query on m^* for any epoch before e^* and no **corrupt** for epoch e^* (Figure 2).

Except for the challenge epoch e^* , we simulate the original game with knowledge of signing keys and key-update tokens. For the challenge epoch e^* , we have to keep the signature from epoch $e^* - 1$ can be updated and verified by pk_{e^*} . Unlike the scheme, we record the signed messages, not their hashes, locally in the global state set $\mathcal{S}$. When queried to **next** in epoch $e^* - 1$, for each message $m_i \in \mathcal{S}$, we first query **Sig**(m_i) to the EUF-CMA challenger of BLS associated with e^* , then compute signature-update token $\Delta_{e^*} := (H(m_i), \sigma_i / \sigma'_i)_{m_i \in \mathcal{S}} \cup pk_{e^*}$, where σ_i, σ'_i is the signature of m_i in epoch $e^*, e^* - 1$ respectively. Similarly, when queried to **next** in epoch e^* , we first generate a fresh key pair (pk_{e^*+1}, sk_{e^*+1}) to sign every $m_i \in \mathcal{S}$ using sk_{e^*+1} , then compute signature-update token $\Delta_{e^*+1} := (H(m_i), \sigma''_i / \sigma_i)_{m_i \in \mathcal{M}} \cup pk_{e^*+1}$, where σ''_i is the signature of m_i in epoch $e^* + 1$. Since BLS-USpt is EFUS, the fresh signature and the updated signature are indistinguishable. If we get a valid forgery $(m^*, \sigma_{e^*}^*, e^*)$ from adversary $\mathcal{A}$, we can output $(m^*, \sigma_{e^*}^*)$ as a forgery to EUF-CMA of BLS.

Theorem 2. *The BLS-USpt from Figure 6 is USpt-EUF-CMA secure.*

¹The BLS scheme can be obtained from modified BLS-based US in Figure 5 by deactivating **Next** and **UpdateS** algorithms, with some trivial changes to remove epochs. We omit the full description for saving space.

Proof. Formally, let us prove it by the following sequence of games, wherein for each game i , the success event is denoted by $S_{A,i}$.

Game 0. This is the original experiment $\text{Exp}_{\text{BLS-USpt},\mathcal{A}}^{\text{uspt-euf-cma}}$. The successful forgery event of $\mathcal{A}$ is denoted by S_0 , where the subscript indicates the game. It holds that

$$\Pr[S_{A,0}] = \Pr\left[\text{Exp}_{\text{BLS-USpt},\mathcal{A}}^{\text{uspt-euf-cma}}(\lambda, n) = 1\right] = \text{Adv}_{\text{BLS-USpt},\mathcal{A}}^{\text{uspt-euf-cma}}(\lambda, n). \quad (2)$$

Game 1. This game is identical to **Game 0** except that we guess an epoch $\hat{e}$ for which $\mathcal{A}$ outputs the forgery in epoch $e^* = \hat{e}$; we abort if $\hat{e} \neq e^*$. Assuming n is the maximum of epochs, and it holds that

$$\frac{1}{n^2} \Pr[S_{A,0}] \leq \Pr[S_{A,1}]. \quad (3)$$

Game 2. This game is identical to **Game 1** except that:

- For call to `next` in epoch $\hat{e} - 1$, we compute $h_i \leftarrow H(m_i)$, $\sigma_{\hat{e},i} \leftarrow \text{BLS.Sig}(sk_{\hat{e}}, m_i)$ for each message $(\hat{e} - 1, m_i, \sigma_{\hat{e}-1,i}) \in \mathcal{S}$, and set $\Delta_{\hat{e}} := (h_i, \sigma_{\hat{e},i}/\sigma_{\hat{e}-1,i})_{m_i \in \mathcal{S}} \cup pk_{\hat{e}}$.
- For call to `next` in epoch $\hat{e}$, we run $(sk_{\hat{e}+1}, pk_{\hat{e}+1}) \leftarrow \text{BLS.Gen}(\lambda)$ to obtain a fresh key pair for epoch $\hat{e} + 1$. For each message $m_i \in \mathcal{S}$, we compute $h_i \leftarrow H(m_i)$ and run $\sigma_{\hat{e}+1,i} \leftarrow \text{BLS.Sig}(sk_{\hat{e}+1}, m_i)$, and set $\Delta_{\hat{e}+1} := (h_i, \sigma_{\hat{e}+1,i}/\sigma_{\hat{e},i})_{m_i \in \mathcal{S}} \cup pk_{\hat{e}+1}$.
- For each call to `sig` for message m in epoch $\hat{e}$, we run $\sigma \leftarrow \text{BLS.Sig}(sk_{\hat{e}}, m)$ and add $(\hat{e}, m, \sigma)$ to $\mathcal{S}$.

Now we claim **Game 2** and **Game 1** are indistinguishable; that is, they are distributed identically. We generate the signature-update tokens by knowing the challenge public key and the messages that need to be updated in advance. Note that we do not have to modify the `update` oracle since the modified tokens $(\Delta_{\hat{e}}, \Delta_{\hat{e}+1})$ can be used for signature valid updates and can be verified by public keys associated to epochs, and the distribution of tokens is identical to the original one. Therefore, it holds that

$$\Pr[S_{A,1}] = \Pr[S_{A,2}]. \quad (4)$$

Now, we claim that we can associate an *EUFCMA* challenger for BLS, and we can construct a valid adversary $\mathcal{B}$ for it. In this case, the challenge key pair is $(sk_{\hat{e}}, pk_{\hat{e}})$ where $\mathcal{B}$ only gets knowledge of $pk_{\hat{e}}$, $\mathcal{B}$ also has the

access to oracle $\text{BLS.Sig}(sk_{\hat{e}}, \cdot)$. Recall that a valid EUF-CMA adversary $\mathcal{B}$ for BLS is not allowed to query $\text{sig}(sk_{\hat{e}}, m^*)$, and $\mathcal{A}$ fails the game if it queried $\text{sig}(m^*, e')$ for message m^* in some epoch $e' \leq \hat{e}$ ($\hat{e} = e^*$) or $\text{corrupt}(e^*)$. Now we need to take care of the situation that $\mathcal{A}$ has all signatures for message m^* for epoch $\hat{e} < e' < n$ and all signing keys except epoch $\hat{e}$ are obtained. We w.l.o.g assume that $\mathcal{A}$ queried $\text{sig}(m^*, \hat{e} + 1)$ since all subsequent signatures can be obtained via updates. Note that before m^* is signed, it will not be added to set $\mathcal{S}$, and no update can be done since the signature-update token Δ does not include such a pair $(H(m^*), \cdot)$ ². For the signing keys, keys $sk_{\hat{e}-1}$ and $sk_{\hat{e}+1}$ are generated independently to $sk_{\hat{e}}$, therefore, they bring no advantage on forge $\sigma_{\hat{e}}^*$.

Now, we can claim that $\mathcal{B}$ is sufficient to act as a valid adversary for an EUF-CMA challenger if $\mathcal{A}$ non-trivially wins the game. Therefore, $\mathcal{B}$ can get from $\mathcal{A}$ and output $(m^*, \sigma_{\hat{e}}^*)$ as a forgery to the EUF-CMA challenger of BLS; that is,

$$\Pr[S_{\mathcal{A},2}] \leq \text{Adv}_{\text{BLS},\mathcal{B}}^{\text{euf-cma}}(\lambda). \quad (5)$$

Putting (2,3,4,5) all together, it holds that,

$$\text{Adv}_{\text{BLS-USpt},\mathcal{A}}^{\text{uspt-euf-cma}}(\lambda, n) \leq n^2 \text{Adv}_{\text{BLS},\mathcal{B}}^{\text{euf-cma}}(\lambda),$$

which concludes this proof. $\square$

5.3. Pros and Cons, and Application

5.3.1. Pros

The proposed USpt will bring many advantages and stronger security guarantees. First of all, the signer does not need to establish a secure channel with the updater or verifier; an authenticated channel is sufficient, which decreases the application system's complexity. Secondly, the signer obtains the real initiative when updating the signatures that he/she really wants to update; that is, assuming the signer still has access to signed messages before the leakage of the signing key, the signer can issue the corresponding signature tokens based on the signed messages (or message-related information, such as their hashes) stored in the medium, not others. Note that the possible loss

²The BLS-USpt is somewhat sesquidirectionally ('sesqui' is Latin for one-and-a-half [19]) updatable: suppose a message is firstly signed in epoch e , one cannot obtain a valid signature in epoch $e' < e$, but it can update the signature to a valid one in epoch $e'' > e$. The US in [4] is bidirectional updatable.

of the signing key is the main reason for updating a key and also the main purpose for applying a US scheme; this threat implies that the attacker may publish some adversarial signatures before the signer is aware of it. In other words, the signer has enough confidence that the signed messages stored in his/her medium are still the legitimate messages he/she signed before. This observation and assumption also grants the signer the ability to distinguish between legitimate signatures and maliciously generated signatures (by the attacker who obtained the outdated signing key; if the signature-token is one-for-all, the attacker can also update its maliciously generated signatures to be valid in the following epoch).

5.3.2. Cons

This construction actually requires the signer to compute and distribute every token of *legitimate* signatures. Here, we refer to *legitimate* signatures as the signatures that were signed by the signer, not the adversary. In this construction, suppose there are n_s signatures to be updated, the signer needs to publish a string $\Delta_{h_{m_1}, e+1} || \Delta_{h_{m_2}, e+1} || \cdots || \Delta_{h_{m_{n_s}}, e+1} || pk_{e+1}$ for an update, which is $n_s + 1$ group elements. Although this string is transferred in the authenticated channel, this is equally to publish the set of newly generated signatures under the updated signing key. The problem is that the signature-update token is linear growth to the number of messages that need to be updated. However, this is not a problem specific to this scheme; this token that relies on legitimate messages requires the ability to identify legitimate messages, and publishing the entire set of legitimate messages is the simplest way to do this. One may use a bloom filter to improve the performance, while the probabilistic nature of the bloom filter makes it inappropriate for cryptographic security requirements.

In addition, this scheme allows the adversarial signer to deny some signatures it signed via updates. This is also an intractable problem. The root of this problem is that in our model, we place more trust in the signer than in a trusted third party. After all, the loss of a private (signing) key represents a loss of identity. The US with secret tokens updates all valid signatures in the previous epoch, no matter the legitimate or adversarial ones, while preserving non-repudiation. We view this difference as a trade-off in trust choice.

5.3.3. Application

The essence of this scheme is to update the signature instead of re-issuing it, which is useful in cases where some signatures have special value. For example, in the NFT trading market, many NFT collections released by artists have great economic value, and buyers use the signatures on these NFTs to verify the true origin of their creators. When an artist's private key is lost, an attacker may release some forged NFTs in his/her name to gain benefits, and the artist's reputation will be damaged, and the NFTs he/she has released may also lose value. After passing the updatable signature, buyers can obtain the artist's update token through other uncompromised authenticated channels (such as some social media) and update the NFT's signature to regain confidence in the NFT they purchased. Instead of re-issuing the NFTs, this scheme saves the interaction between the artist and buyer and many other potential costs, such as gas consumption caused by re-issuance.

6. Conclusion

This work aims to publicise the signature-update token for US schemes. First, we start with the security model and consider the security definitions that can be achieved after making the token public. Then, based on the construction of the existing US schemes, we discuss some possible construction directions: construction without the help of the transform function f and construction of one-way f^{-1} . We prove that the first idea is denied and then construct the USpt scheme based on BLS by bypassing the impossible result from existing work.

6.1. Future Work

The main drawback of our scheme is bandwidth consumption during signature updates. This is linear growth with the number of total signatures. Since the direct security demand is to authorize verifiers to distinguish legitimate signatures from adversarial ones, this scheme publishes the legitimate hash list for verification. We will keep working on this problem to find a more efficient solution. A convenient way is to broaden the assumptions, which also leads to another future work of ours, to set up an NFT transaction scenario for this scheme and implement it and related assumptions in smart contracts and blockchain oracles. As far as we know, the current NFT third-party trading market is not responsible for the loss of users' private keys, which

is technically the responsibility of the crypto wallet. However, considering the amount of money involved in the NFT market, we may compromise to provide certain protection for NFT traders.

We have not yet explored the possibility of publicising tokens in the lattice-based US. This is a major work for the future.

References

- [1] K. Cohn-Gordon, C. Cremers, L. Garratt, On post-compromise security, in: 2016 IEEE 29th Computer Security Foundations Symposium (CSF), IEEE, 2016, pp. 164–178. 1
- [2] B. LaMacchia, K. Lauter, A. Mityagin, Stronger security of authenticated key exchange, in: International conference on provable security, Springer, 2007, pp. 1–16. 1
- [3] H. Krawczyk, Hmqv: A high-performance secure diffie-hellman protocol, in: Annual international cryptology conference, Springer, 2005, pp. 546–566. 1
- [4] V. Cini, S. Ramacher, D. Slamanig, C. Striecks, E. Tairi, Updatable signatures and message authentication codes, in: IACR International Conference on Public-Key Cryptography, Springer, 2021, pp. 691–723. 1, 1.1, 1.2, 2.2, 2.2, 3.1, 3.2.1, 3.2.2, 4.2.1, 5.1, 2
- [5] R. Canetti, Universally composable security: A new paradigm for cryptographic protocols, in: Proceedings 42nd IEEE Symposium on Foundations of Computer Science, IEEE, 2001, pp. 136–145. 1
- [6] D. Derler, D. Slamanig, Key-homomorphic signatures: definitions and applications to multiparty signatures and non-interactive zero-knowledge, Designs, Codes and Cryptography 87 (2019) 1373–1413. 1.2, 2.2, 4.3
- [7] D. Boneh, B. Lynn, H. Shacham, Short signatures from the weil pairing, in: International conference on the theory and application of cryptology and information security, Springer, 2001, pp. 514–532. 1.2
- [8] A. Lehmann, B. Tackmann, Updatable encryption with post-compromise security, in: Advances in Cryptology–EUROCRYPT 2018:

- 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29-May 3, 2018 Proceedings, Part III 37, Springer, 2018, pp. 685–716. 1.2
- [9] J. Zhou, Z. Liu, An improved updatable signature scheme with weakened token, in: International Conference on Frontiers in Cyber Security, Springer, 2023, pp. 419–438. 1.2
 - [10] W. Diffie, M. E. Hellman, New directions in cryptography, in: Democratizing Cryptography: The Work of Whitfield Diffie and Martin Hellman, 2022, pp. 365–390. 1.2
 - [11] B. Barak, O. Goldreich, R. Impagliazzo, S. Rudich, A. Sahai, S. Vadhan, K. Yang, On the (im) possibility of obfuscating programs, in: Annual international cryptology conference, Springer, 2001, pp. 1–18. 1.2
 - [12] A. Jain, H. Lin, A. Sahai, Indistinguishability obfuscation from well-founded assumptions, in: Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing, 2021, pp. 60–73. 1.2
 - [13] M. Bellare, S. K. Miner, A forward-secure digital signature scheme, in: Annual international cryptology conference, Springer, 1999, pp. 431–448. 1.2
 - [14] T. V. Doan, Y. Psaras, J. Ott, V. Bajpai, Toward decentralized cloud storage with ipfs: opportunities, challenges, and future considerations, *IEEE Internet Computing* 26 (6) (2022) 7–15. 1.3
 - [15] Q. Wang, R. Li, Q. Wang, S. Chen, Non-fungible token (nft): Overview, evaluation, opportunities and challenges, *arXiv preprint arXiv:2105.07447* (2021). 1.3
 - [16] D. Derler, D. Slamanig, Key-homomorphic signatures: Definitions and applications to multiparty signatures and non-interactive zero-knowledge, *Cryptology ePrint Archive*, Paper 2016/792, <https://eprint.iacr.org/2016/792> (2016). doi:10.1007/s10623-018-0535-9. URL <https://eprint.iacr.org/2016/792> 2.2
 - [17] D. Boneh, C. Gentry, B. Lynn, H. Shacham, et al., A survey of two signature aggregation techniques (2003). 5.1

- [18] D. Boneh, C. Gentry, B. Lynn, H. Shacham, Aggregate and verifiably encrypted signatures from bilinear maps, in: *Advances in Cryptology—EUROCRYPT 2003: International Conference on the Theory and Applications of Cryptographic Techniques*, Warsaw, Poland, May 4–8, 2003 Proceedings 22, Springer, 2003, pp. 416–432. 5.1
- [19] B. Poettering, P. Rösler, Towards bidirectional ratcheted key exchange, in: *Advances in Cryptology—CRYPTO 2018: 38th Annual International Cryptology Conference*, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part I 38, Springer, 2018, pp. 3–32. 2